Let’s write a much simpler version of file watcher to see how watching and signals go together. This `SimplifiedFileWatcher` logs a `Terminated` message when the log processor explicitly stops, not because of an exception. The file watcher can watch any actor included in the `Watch` message and receives the `Terminated` signal if a watched actor stops.

Listing 5.1 `Watcher` actor

```
import akka.actor.typed.{ ActorRef, Behavior, ChildFailed, Terminated }
import akka.actor.typed.scaladsl.{ Behaviors }

object SimplifiedFileWatcher {

  sealed trait Command
  final case class Watch(ref: ActorRef[String]) extends Command

  def apply(): Behavior[Command] =
    Behaviors
      .receive[Command] { (context, message) =>
        message match {
          case Watch(ref) =>
            context.watch(ref)                  ❶
            Behaviors.same
        }
      }
      .receiveSignal {
        case (context, Terminated(ref)) =>      ❷
          context.log.info("terminated")
          Behaviors.same
      }
}
```

❶ Watches any actor, whether a child or not

❷ Deals with the situation when the watched actor stops

You can prove that this works as expected with the following test. To do so, you define a log processor that stops when it receives a "stop" message and treats any other string message as unhandled, which